

DRL-based Multi-Stream Scheduling of Inference Pipelines on Edge Devices

Danny Pereria*, Sumana Ghosh†, Soumyajit Dey*

*Indian Institute of Technology Kharagpur, India, † Indian Statistical Institute Kolkata, India
dannyjpereira@kgpian.iitkgp.ac.in, soumya@cse.iitkgp.ac.in, sumana@isical.ac.in

Abstract—Real-time scheduling of multiple neural network-based inference pipelines on Graphics Processing Unit (GPU) based edge devices is an active area of research nowadays. Applications like Advanced Driver-Assistance Systems (ADAS) execute multiple such inference pipelines to make informed decisions on driving scenarios. The real-time performance of ADAS is often limited by platform resource limitations and thus incurs execution latency which ultimately leads to deadline violation. In this regard, modern GPUs provide support for concurrent execution of multiple compute streams. However, there is a lack of scheduling strategies in the literature that consider multiple such compute streams and focus on the concurrent execution of inference pipelines for more efficient real-time scheduling. In this paper, we address this issue by proposing a Deep Reinforcement Learning (DRL) based solution for *multi-stream scheduling* of inference pipelines on edge GPUs. Using DRL, we learn how to map every layer of the target inference pipelines to high or low-priority streams, while satisfying task-level deadline requirements. The experimental evaluation shows the efficacy of the proposed approach as compared to some baseline approaches.

Index Terms—Deep Reinforcement Learning, Edge Device, GPU, Real-Time Scheduling.

I. INTRODUCTION

Perception-based computing is an indispensable part of today's autonomous systems. One such example is Advanced Driver-Assistance Systems (ADAS). ADAS supports various time-critical features like lane departure warnings, adaptive cruise control, etc., to ensure better road safety. The underlying perception computing task employs multiple Neural Network (NN)-based real-time inferencing pipelines for object detection. To achieve timely completion of these object detection tasks, modern ADAS systems often co-schedule them in embedded GPUs. Such task mappings further demand completion within the deadline. However, scheduling these tasks on the GPU of an edge device while minimizing deadline misses is challenging since modern inference pipelines often rely on resource-and-time-intensive Convolutional Neural Networks (CNNs) for object detection.

To reduce the overall execution latency of these pipelines, recent research efforts emphasize the following two concepts. 1) Overlapping memory and compute operations: that means, while some computations of one pipeline are being performed, the input data (i.e., weight tensors) of another pipeline is simultaneously loaded from memory. 2) Overlap compute operations of multiple pipelines: this is done by allowing certain

kernel computations to run concurrently. A notable approach that follows the above concepts is proposed in [1]. The authors propose a Satisfiability Modulo Theory (SMT)-based optimal scheduling scheme to explore different mappings of memory and compute operations within the inference pipelines, taking advantage of multiple CUDA *streams* [2]. Note that a CUDA stream is a queue of GPU operations scheduled to execute in a specific sequence. Generally, modern GPUs have the necessary support for these CUDA streams. While beneficial for smaller task setups, the scalability of this method becomes a serious concern as the pipeline size and complexity grow. On the other hand, following the second concept, the authors in [3] present multi-streamed execution of Real-time (RT) tasks and Best-Efforts (BE) tasks. RT tasks are always scheduled on high-priority streams, whereas BE tasks are scheduled on low-priority streams. This approach has a fixed setup for the task execution on two streams, whereas, in our approach, we identify the optimal mapping of operations to streams following a multi-stream setting.

More generically, in contrast to all these existing approaches, we address this issue by introducing a novel Deep Reinforcement Learning (DRL)-based technique that employs multiple agents interacting with the environment in order to efficiently schedule multiple inference pipelines. The salient features of the proposed DRL-based technique are as follows.

- 1) We design a custom task scheduling environment and train multiple RL agents using Deep-Q-Learning (DQN) to produce optimized multi-stream schedules for a set of inference pipelines.
- 2) We incorporate operation level co-schedulability awareness in the RL agents using suitable reward functions that incentivize minimization of deadline misses.
- 3) We validate the usefulness of the proposed approach using real-life object detection pipelines and obtain better performance over existing baselines [1], [4].

II. RELATED WORK

Existing research efforts towards latency minimization of ADAS pipelines on embedded GPGPUs are as follows. To co-schedule multiple CNN tasks, the S^3DNN framework [4] proposes a multi-streamed Least Slack First (LSF) algorithm adapted for GPU kernel execution. In [3], the authors present a framework called DART to schedule real-time and best-effort tasks on the CPU-GPU platform. On the other side, LaLaRAND [5] improves real-time CNN task scheduling with CPU-friendly quantization and layer-level CPU/GPU

This research was conducted under AICTE's QIP scheme. The authors express gratitude for the valuable inputs from Anirban Ghose and Sumanta Dey in this research endeavour.

allocation, enhancing the overall efficiency. In all of the above-mentioned approaches, it is assumed that the weight tensors are resident in memory and available for CPU/GPU computation. As shown in [1], such an assumption is not viable for a time-varying demand of weight tensors based on the road condition and is also not feasible for resource-constrained edge devices. To overcome it, the authors in [1] propose an SMT-based approach that opportunistically loads/discards weight tensors based on requirement while co-scheduling relevant compute operations belonging to different NN pipelines. Though it bestows optimal multi-stream scheduling, as mentioned earlier, this work suffers from serious scalability issues with the growing size and complexity of inference pipelines.

Though RL has been widely utilized in developing game-playing agents such as AlphaGo, Backgammon, etc., there exist very few applications of RL algorithms in scheduling problems. In [6], a multi-agent deep RL approach is proposed to schedule multiple tasks on a multiprocessor system. Similarly, in [7], the authors develop an RL-based approach to schedule workload on distributed compute clusters. In [8], an RL agent is trained using DQN to decide upon CPU and GPU mapping of tasks to deliver reliable and predictable performance of inference pipelines. As quite evident from the above discussion, the focus and setup of our work are very different from these existing RL-based approaches.

III. SYSTEM MODEL AND MOTIVATION

System Model: Let $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_n\}$ be the set of n tasks, $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ be the set of periods of these n tasks, $\mathcal{D} = \{D_1, D_2, \dots, D_n\}$ be the relative deadlines, and $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$ be the Worst-Case Execution Time (WCET) of n tasks. We consider a task configuration as the tuple (τ_i, p_i, D_i, c_i) where $\tau_i \in \mathcal{T}, p_i \in \mathcal{P}, D_i \in \mathcal{D}$, and $c_i \in \mathcal{C}$ and a set of task configurations as $(\mathcal{T}, \mathcal{P}, \mathcal{D}, \mathcal{C})$. Each task τ_i is represented as a Directed Acyclic Graph (DAG) of the form, $G = (V, E)$, where V denotes the nodes (i.e., operations over the layers within a task) and E denotes directed edges (i.e., dependencies) among the nodes. The operations are broadly categorized as *compute* and *memory* operations, where compute operations mainly indicate the execution of kernels on GPU and memory operations indicate data transfer between the CPU host and the GPU device. Given a period of each task, we calculate the hyper-period of the task configuration as $\mathcal{H} = lcm(p_1, p_2, \dots, p_n)$. The total number of jobs per task within hyper-period $\mathcal{H}$ is represented as set $J = \{j_1, j_2, \dots, j_n\}$, where $j_i = \lfloor \mathcal{H}/p_i \rfloor$. We use the representation (i, k, x) to denote the x -th operation of the k -th job of the i -th task τ_i . The variables $a_{i,k}$ and $d_{i,k}$ are used to denote the arrival time and the absolute deadline of the k -th job of τ_i respectively. The value of $d_{i,k}$ is given by $d_{i,k} = a_{i,k} + D_i$, where D_i is the relative deadline of τ_i . Since each task τ_i has to finish its execution before $d_{i,k}$ and each job of τ_i has $|V|$ operations, it is essential that each operation to be finished within its local deadline. An upper bound on the local deadline of individual operation can be calculated as

$$ld_{i,k,x} \leq (k+1) * d_{i,k} - \sum_{r=x+1}^{|V|-1} c_{i,k,r}, \text{ where } c_{i,k,r} \text{ represents}$$

profiled WCET of (i, k, r) -th operation. We use the variable $ST_{i,k,x}$ and $FT_{i,k,x}$ to represent the start time and the finish time of (i, k, x) -th operation, respectively.

We assume the availability of four resources for the execution of jobs on the target device, like modern embedded GPGPUs. The set of resources is, $\mathcal{R} = \{cs_h, cs_l, h2d, d2h\}$ where, cs_h and cs_l represents high- and low-priority compute streams respectively, $h2d$ and $d2h$ represents memory streams for data transfer operations from host-to-device and device-to-host respectively. Since a schedule is a mapping from operations to streams, given a task configuration (τ_i, p_i, D_i, c_i) and the set of streams $\mathcal{R}$, we formalize the notion of a *multi-stream schedule* as, $\mathbb{S} = \{ \langle (i, k, x), ST_{i,k,x}, FT_{i,k,x}, stream_{i,k,x} \rangle \}$ such that $1 \leq i \leq n, 0 \leq k < j_i, 0 \leq x < |V|$ and $stream_{i,k,x} \in \mathcal{R}$.

Motivational Example: In order to motivate the proposed

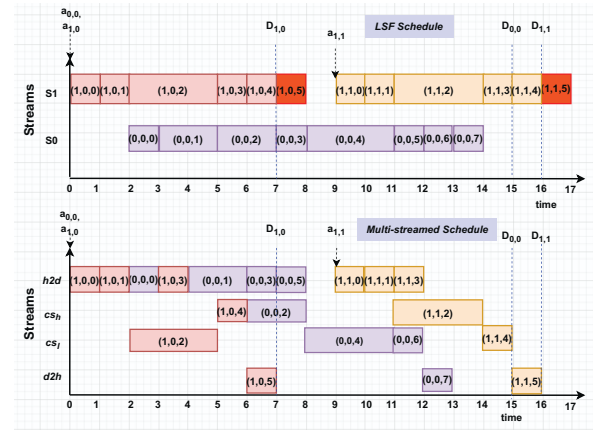


Fig. 1: Least Slack First vs Multi-streamed scheduling

idea, we consider a representative example with two tasks, $\{\tau_0, \tau_1\}$. The task τ_0 has one job, and τ_1 has two jobs. The deadlines of each job of τ_0 and τ_1 are 15 ms and 7 ms respectively. We assume that the first job of both τ_0 and τ_1 arrives at time 0 ms and the second job of τ_1 arrives at 9 ms.

We implement two approaches to get the schedule of this task configuration. The first approach is the existing LSF algorithm [4]. The schedule obtained following LSF is shown in the top half of Figure 1. It uses two streams, S_0 for mapping operations of jobs of task τ_0 and S_1 for mapping operations of jobs of task τ_1 . However, as evidently seen from the figure, this approach fails in meeting the deadline of τ_1 as both of its jobs are missing their respective deadlines (highlighted by the red colour). The reason for this is the lack of overlapping between memory-compute and compute-compute operations across the streams.

The second approach incorporates dedicated streams, i.e., $h2d$ and $d2h$ for memory operations and cs_h and cs_l for compute operations. This is how it creates room for better overlapping of compute-compute, memory-compute, and memory-memory operations. This results in an improved schedule as shown in the bottom half of Figure 1 where both tasks meet their respective deadlines. The use of separate streams for

specific types of operations enables more efficient and timely execution of all the jobs, hence leading to the successful execution of all the tasks satisfying their deadlines.

Problem Statement: Given a set of task configurations as $(\mathcal{T}, \mathcal{P}, \mathcal{D}, \mathcal{C})$ and the set of streams $\mathcal{R}$, our objective is to synthesize an *optimal multi-stream schedule* $\mathcal{S}$ such that all tasks meet their respective deadlines as fast as possible. In other words, we aim to synthesize the start and finish times of all operations in all jobs of all tasks while efficiently mapping compute operations to high/low priority compute streams and memory operations to memory streams such that the number of deadline misses gets minimized. Towards this goal, we employ a Deep Reinforcement Learning (DRL)-based technique, where multiple RL agents are trained to learn an optimal policy function that guides them in generating an optimal multi-stream schedule.

IV. PROPOSED DRL-BASED METHODOLOGY

Our methodology for training suitable DRL agents that perform optimized operations to stream mapping is as follows.

A. Designing RL Agents

We have designed three RL agents for three streams; namely, *compute agent*, *d2h agent*, and *h2d agent*. The goal of these agents is to map the current operation to the respective stream to minimize the deadline misses. The schematic diagram of RL agents and custom task scheduling environment is depicted in Figure 2.

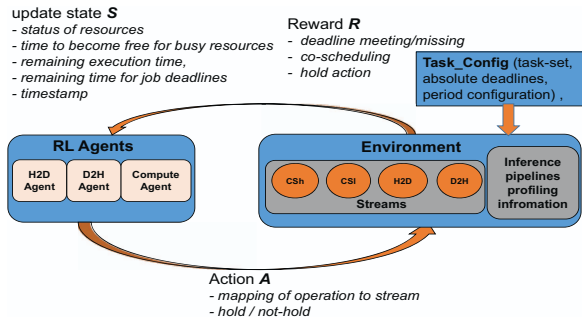


Fig. 2: Design of multi-agent RL

State Representation: For a given set of task configurations $(\mathcal{T}, \mathcal{P}, \mathcal{D}, \mathcal{C})$, the state S of an RL agent consists of the following attributes:

- 1) $cs_{hf_s}, cs_{lf_s}, h2d_{fs}, d2h_{fs}$: These variables are used to capture the free status of a particular stream at the current timestamp. If the stream is free, it is denoted as 1, otherwise by 0. E.g., if the stream cs_h is free, cs_{hf_s} will be set to 1, otherwise to 0.
- 2) $cs_{ttf}, cs_{lff}, h2d_{ttf}, d2h_{ttf}$: These variables are used to store the time amount after which a particular stream will be free, from the current timestamp.
- 3) $rex_0, rex_1, rex_2, \dots, rex_n$: These variables are used to capture the remaining execution times of n jobs.
- 4) $rttd_0, rttd_1, rttd_2, \dots, rttd_n$: These variables are used for tracking the remaining time to deadline for n jobs.

- 5) *hyper-period, timestamp*: The variable *hyper-period* is used to store the hyper-period of the given task configuration and the variable *timestamp* is used to store the current timestamp value.

Action Selection: Given the state S and the current operation op , the mapping function $\mathcal{M}$ employed by the RL agent is given as $\mathcal{M}(S, op) = action$. Here, the values for the *action* are chosen based on the different operation-to-stream mapping scenarios. We set three values, i.e., 0, 1 and 2, to the *action* of each RL agent. For the *compute agent*, the value 0/1 indicates that the operation is mapped to cs_h/cs_l stream, respectively, whereas the value 2 indicates to *hold* the mapping, which means, the agent does not map the operation to any stream in the current timestamp, rather it holds back the operation. Similarly, in the case of *h2d* and *d2h agent*, the value 0 and 1 indicates that the operation is mapped to $h2d/d2h$ stream, respectively, whereas value 2 represents the *hold* action. Note, the *hold* action helps the RL agents to delay the scheduling of an operation due to co-schedulability constraints.

B. Constructing the Reward Function

During the training, an action is selected based on the average reward value, which quantitatively measures the performance of the agent against the chosen action. We devise the reward function based on the following criteria.

- 1) *Job Deadline*: Meet the deadline of a job and get the reward, otherwise, suffer the penalty – this principle guides us to set the reward value as follows. If a job completes its execution before its specified deadline, a reward of +1 will be given to all the operations of that job. If the deadline is missed, a penalty of -1 will be given to all the operations of that job. This steers the agent to choose all the actions pertaining to completed jobs, so the rewards for all operations of the currently finished jobs are updated accordingly.
- 2) *Operation Deadline*: As early as the deadline of an operation is met, a better reward will be given. Hence, if the current operation (i, k, x) scheduled by the RL agent meets its local deadline much earlier, then a reward of e^{-t} (positive value in this case) is given, where t is the difference between the operation's finish time and its local deadline. If the operation finishes going beyond its local deadline, then the value of e^{-t} (negative value in this case) will act as a penalty.
- 3) *Co-Schedulability*: Since co-schedulability enhances parallel execution of tasks, it leads to faster task completion and thus lesser deadline violations. Therefore, allow more co-schedulability to get more rewards. If an agent correctly infers the co-schedulability of compute-compute operations across the cs_h and cs_l streams, then the reward value will be updated with +1 per overlapping, otherwise 0. During the decision, the environment consults the *co-schedulability table* which provides information about which compute kernels of one pipeline are co-schedulable with compute kernels in another pipeline.

- 4) *Hold Action*: If an agent chooses the hold action for an operation, its reward value will be updated with e^{-t} , where t is the difference between the operation's finish time and its local deadline. Since each hold action is refraining the operation to be executed at the current timestamp, and thereby incurring a delay towards its deadline, the reward value should be inversely proportional to the delay. Hence, the agent will try to skip the hold action until it is inexorable due to some other constraint like co-schedulability.

If an agent chooses to perform a *hold action*, it will receive a reward based only on the hold action. The *compute agent* will receive a total reward consisting of both co-schedulability and operation deadline rewards. In contrast, the *h2d agent* receives a reward solely based on the operation deadline, while the *d2h agent* is rewarded based on the job deadline since the *d2h* operation is the last operation within the inference pipeline. Therefore, the finish time of this operation has an impact on the successful completion of the job satisfying its deadline.

C. Designing the Custom Task Scheduling Environment

The scheduling environment has several goals, including executing actions provided by agents, updating its states, and providing a reward signal to agents. To facilitate this, the environment maintains three priority queues: *h2d_readyQ*, *d2h_readyQ*, and *cs_readyQ* to keep track of host-to-device, device-to-host memory operations and compute operations, respectively. When a job arrives, it is assumed that all of its memory operations are ready, and they are added to the *h2d_readyQ* queue. A compute operation becomes ready when its corresponding memory operation and the preceding compute operation are finished. Once this condition is met, the compute operation is added to the *cs_readyQ*. When the last compute operation of a job completes, it makes the last *d2h* memory operation ready, and it is added to the *d2h_readyQ* queue. Operations within the ready queues are arranged in ascending order of their upper bound on local deadlines.

The environment tracks mapped operations using four lists: *m_csh*, *m_csl*, *m_h2d*, and *m_d2h*. The *compute agent* handles *m_csh* and *m_csl*, while the *h2d* and *d2h* agents manage *m_h2d* and *m_d2h*, respectively. The variable *timestamp* keeps track of time point values, while the variable *current_op* stores the operation currently under consideration during the schedule generation process. At any given time point, the environment checks the ready queues for operations. When multiple ready queues are non-empty, RL agents are invoked for all ready operations. Note that only one *h2d* operation can be dequeued due to one available *h2d* stream. However, multiple compute operations are dequeued. This is because there are two compute streams, allowing parallel execution. It leverages co-schedulability by mapping compute operations to different streams while adhering to constraints.

Figure 3a) shows the initial configuration of the environment at *timestamp* value equals 0. It is assumed that two jobs arrived at this point and each job has a total of eight operations. For simplicity it is assumed that the first and subsequent odd

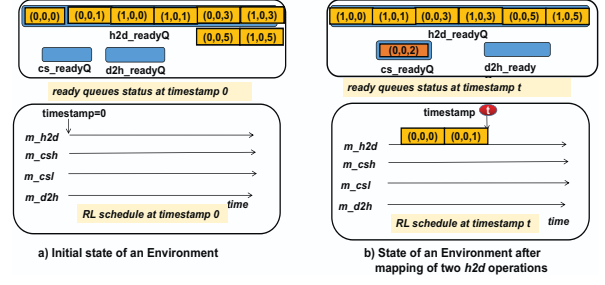


Fig. 3: Initial state of an Environment

indexed operation is a memory operation; even indexed operation is a compute operation. As in Figure 3a), at *timestamp* = 0, all the memory operations of the first and second jobs are added in *h2d_readyQ* (except the last memory operation). Due to dependency on memory operations, neither job's compute operations is ready. Figure 3b) depicts the scenario when the first two *h2d* operations are mapped to *h2d* streams by the *h2d agent*, and the *timestamp* is increased at t . Note that since the dependency of the compute operation (0,0,2) is satisfied, the same has been added in *cs_readyQ*. Since both *h2d_readyQ* and *cs_readyQ* have ready operations, our environment calls respective agents to get actions pertaining to these operations.

Figure 4 illustrates the changes in the scheduling environment after mapping certain operations at four different time points. At time point A, the *h2d* operation (0,0,3) and the compute operation (1,0,2) are ready. According to the mapping diagram, the *compute agent* maps the compute operation (1,0,2) to *cs_l* stream, while the *h2d agent* holds the memory operation (0,0,3). Moving to time point B, the memory operation (1,0,3) completes. Since the dependent compute operation (1,0,2) and the memory operation (1,0,3) have finished earlier, the compute operation (1,0,4) becomes ready and is added to *cs_readyQ*. The *compute agent* maps it to *cs_l* stream, respecting co-schedulability constraints.

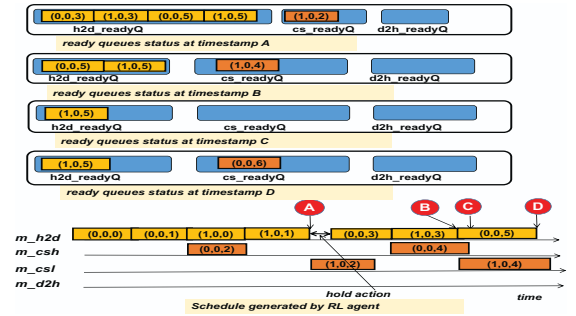


Fig. 4: State of an Environment at timestamps A, B, C, D

At time point C, the compute operation (0,0,4) finishes. However, at this point, the compute operation (0,0,6) does not become ready because it depends on the memory operation (0,0,5), and the operation (0,0,5) is not over yet. Once the *timestamp* is reached at time point D, the compute operation (0,0,6) becomes ready, and the environment triggers the

agents to provide actions for the compute operation (0, 0, 6) and the memory operation (1, 0, 5). When the timestamp value reaches the finish time of (0, 0, 6), the last operation (0, 0, 7) becomes ready and is added to the $d2h_readyQ$. Subsequently, the $d2h$ agent provides the necessary action.

The Low-Level Scheduler: A low-level scheduler has been developed in CUDA C++ to execute schedules generated by DRL agents on the Jetson Xavier NX board. This scheduler is responsible for coordinating the execution of the provided schedule on the end device, dispatching operations based on their timing information, and mapping to specific streams. Additionally, it handles tasks related to stream and memory management.

V. EXPERIMENTS AND RESULTS

A. Target Platform: We have tested our RL-generated schedules on NVIDIA's Jetson Xavier NX embedded board. The Jetson Xavier NX has an NVIDIA Carmel ARMv8.2 64-bit CPU with six cores grouped into three clusters. Also, it has an integrated GPU with 384 CUDA cores and 48 Tensor cores. The board has 8GB 128-bit LPDDR4x DRAM. It is equipped with two copy engines for bi-directional data transfers.

B. Object Detection Inference Pipelines (Tasks): For our experiments, we have considered two well-known and widely used object detection CNN inference pipelines: YoloLite [9] and tinyYolov2 [10]. YoloLite inference pipeline consists of 13 layers with seven convolution layers, five max pool layers, and one region layer, whereas tinyYolov2 has 16 layers with nine convolution layers, six max pool layers, and one region layer. For our experiments, we have chosen these two inference pipelines because of their applicability in resource-constrained embedded devices due to their lightweight nature [11], [12].

C. Task Configuration Generation and Training: For training RL agents, we have considered a task set for four tasks $\mathcal{T} = \{\tau_0, \tau_1, \tau_2, \tau_3\}$ with τ_0, τ_2 as yoloLite inference pipeline tasks and τ_1, τ_3 as tinyYolov2 inference pipeline tasks. We have generated a set of multiple task configurations, $(\mathcal{T}, \mathcal{P}, \mathcal{D}, \mathcal{C}) = \{(\tau_i, p_i, D_i, c_i)\}$, with these four tasks by (i) varying the period of YoloLite tasks between 20ms to 200ms and the period of tinyYolov2 tasks between 30ms to 200ms and (ii) setting WCET (i.e., c_i) in a way such that the utilization remains between 0.5 to 3.0. Note, the utilization of a set of task configurations with n tasks is calculated as, $U_{(\mathcal{T}, \mathcal{P}, \mathcal{D}, \mathcal{C})} = \sum_{i=0}^{n-1} c_i/p_i$ where $c_i \in \mathcal{C}$, $p_i \in \mathcal{P}$. We have considered 580 such task configurations for training RL agents. The RL agent has undergone 100 training epochs to better generalize using the well-known Deep-Q-Network (DQN) algorithm. Hyper-parameters used for DQN training are as follows: memory capacity of replay buffer: 1000, batch size: 32, policy NN layers: MLP (two layers of 64). Training and testing datasets are kept different.

D. Baseline Algorithms: We have considered *multi-streamed Least Slack First (LSF)* algorithm introduced in [4] as our first baseline algorithm. The authors in [4] have modified the basic version of LSF into a multi-streamed version to suit kernel execution on a multi-streamed CUDA environment, wherein the ready kernels from multiple inference tasks are

TABLE I: Comparison between Multi-streamed LSF and Multi-streamed RL approaches: Deadline misses

Exp. No.	Task with #job	Task Periods	Hyper-period	Utilization	MS-LSF	MS-RL
1	[2, 3, 3, 2]	[120, 80, 80, 120]	240	0.83	0	0
2	[4, 4, 3, 4]	[90, 90, 120, 90]	270	0.84	0	0
3	[2, 1, 1, 2]	[60, 120, 120, 60]	120	1.0	0	0
4	[3, 1, 1, 2]	[40, 120, 120, 60]	120	1.125	1	0
5	[4, 3, 4, 3]	[60, 80, 60, 80]	240	1.125	10	0
6	[5, 2, 4, 4]	[40, 100, 50, 50]	200	1.42	13	0
7	[1, 1, 2, 2]	[80, 80, 40, 40]	80	1.5	3	0
8	[3, 3, 1, 1]	[30, 30, 90, 90]	90	1.77	7	3
9	[3, 2, 3, 3]	[40, 60, 40, 40]	120	1.79	11	4
10	[1, 3, 1, 3]	[90, 30, 90, 30]	90	2.0	6	6

scheduled on multiple streams as per the *slack time*. The slack time denotes the amount of time left to reach the task deadline considering the execution completed so far. The second baseline approach considered is the Satisfiability Modulo Theories (SMT)-based approach presented in [1]. Since this generates optimal scheduling and also considers a multi-streamed CUDA environment for kernel executions, we have chosen this approach for the comparison to check how close to optimal our proposed RL-based scheduling is.

E. Comparison with LSF Approach: To assess the performance of the RL agent compared to the baseline algorithms, we have tested the trained RL agents on various task setups. We have considered 10 representative task setups for testing with varying utilization factors. The experiments are listed in ascending order on their utilization factors (i.e., Column 5) in Table I. Column 2 of the table indicates the job counts while Column 3 provides the period of the four tasks. Columns 6 and 7 indicate the number of tasks that miss their deadlines when the multi-streamed LSF (MS-LSF) algorithm and the multi-streamed RL approach (MS-RL) are used for scheduling, respectively. Due to resource constraints on the edge device, we have limited the total number of jobs to 15. The time required to produce the schedules in Table I is less than 1s for the MS-RL approach. Since this is quite prompt in giving the results, enhancing the method with a larger task set and hyper-periods is the immediate next step.

Real-Time Performance Analysis: The last column of Table I reveals a significant reduction in the number of tasks missing their deadlines when the proposed RL-based approach is used for scheduling. This improvement can be attributed to the RL agent's ability to effectively learn from sample data and optimize schedules by overlapping memory-compute and compute-compute operations. By leveraging its learned knowledge, RL agents can generate schedules that meet the timing requirements of all the tasks quite better, hence reducing the number of deadlines misses. Additionally, Figure 5a) displays the average lateness values for the 10 experiments listed in Table I. Note that lateness refers to the amount of time by which a job misses its deadline; hence, if the task meets the deadline, then the lateness will be a negative

quantity, otherwise, a positive quantity. So, for a set of n tasks, as fast as their deadlines are met, the more negative value we will get as the average lateness value. The graph illustrates a trend of increasing average lateness values as the utilization factor increases with the increasing experiment numbers. It is quite justified that increasing the utilization allows more jobs to be finished within a given hyper-period. This in turn increases the amount of time by which jobs can miss their deadlines, thereby going towards a more positive quantity as a whole. However, it is worth noting that our proposed RL-based approach consistently outperforms the baseline LSF approach in terms of average lateness values, i.e., faster completion of tasks w.r.t their deadlines as compared to LSF with the increasing utilization factors.

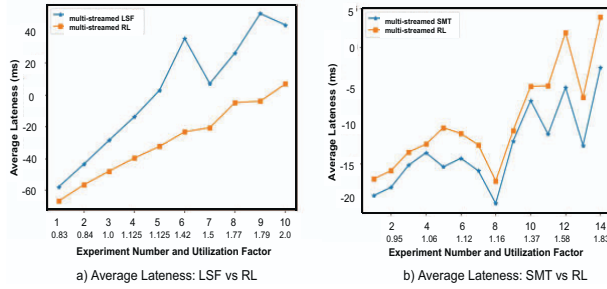


Fig. 5: Average Lateness Comparison

F. Comparison with SMT-based Approach: Table II reports the comparative results where the sixth and seventh columns present the number of deadline misses obtained by running the SMT-based and the RL-based scheduling approaches, respectively. Upon analysis, it is observed that in 11 out of 14 experiments, the RL-based scheduling performs as well as the SMT-based scheduling (the same number of deadline misses, all 0). However, for exp. no. 12, 13, and 14, SMT-based scheduling beats RL-based scheduling but with a little margin, i.e., (1 vs. 4), (1 vs. 3), and (1 vs. 3) respectively.

Figure 5 b) reports the comparative results in terms of average lateness. As it is evident from the figure that the average lateness values obtained following RL-based scheduling are very close to the values obtained by SMT-based scheduling.

Efficiency in Schedule Generation: As mentioned earlier, the SMT-based approach suffers from the scalability issue which can also be witnessed in Column 8 of Table II. It clearly indicates that a significant amount of time (in the range of hours) is required to generate schedules, even though the approach runs in an *offline manner*. More surprisingly, it does not scale (becomes unresponsive) even for a set of 4 task configurations. All the related experiments were run on a server machine with 24-core Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GH with 64GB RAM. In contrast, under the same machine setup, the RL-based approach shows more efficiency. Column 9 of Table II reports schedule generation time in the milliseconds range for the RL-based approach. Although it requires an initial training process of 23 h (approx.), once the RL agents are trained, on deploying they quickly produce

TABLE II: Comparison between SMT-based and RL-based approaches: Deadline Misses, Timing Performance

Exp. ID.	Task with #job	Task Periods	Hyper-period	Utilization	SMT DM	RL DM	SMT (hrs)	RL (ms)
1	[7, 1]	[20, 140]	140	0.92	0	0	46.03	246.8
2	[6, 1]	[20, 120]	120	0.95	0	0	19.11	188.3
3	[5, 1]	[20, 100]	100	1.0	0	0	10.80	141.0
4	[4, 1]	[20, 80]	80	1.06	0	0	4.24	95.22
5	[1, 2]	[60, 30]	60	1.08	0	0	0.79	43.56
6	[4, 3]	[30, 40]	120	1.12	0	0	232.4	117.3
7	[3, 1]	[20, 60]	60	1.16	0	0	1.95	58.68
8	[2, 1, 1]	[30, 60, 60]	60	1.16	0	0	46.74	43.74
9	[3, 4]	[40, 30]	120	1.20	0	0	114.0	112.2
10	[2, 1]	[20, 40]	40	1.37	0	0	0.89	36.20
11	[2, 1, 2]	[30, 60, 30]	60	1.41	0	0	182.8	55.62
12	[3, 2]	[20, 30]	60	1.58	1	4	53.77	57.84
13	[2, 2, 1]	[30, 30, 60]	60	1.58	1	3	221.1	58.22
14	[1, 1, 1]	[30, 30, 30]	30	1.83	1	3	8.23	33.01

schedules *on-the-fly*. In summary, while the SMT-based approach struggles with longer run times and scalability issues, the RL-based approach generates schedules significantly faster once the agents are trained. This makes the RL-based approach more suitable for dynamic scheduling scenarios where new tasks need to be accommodated quickly and efficiently.

VI. CONCLUSION AND FUTURE WORK

This work develops a novel DRL-based approach for multi-stream scheduling of inference pipelines on edge devices. The effectiveness of the proposed approach has been evaluated with respect to the state-of-the-art methods and also validated in a real-world setup. In the future, we plan to extend this setup to incorporate memory awareness, ensuring the multi-stream schedule adheres to user-supplied memory constraints. Addressing the issue of memory contention from concurrent access to shared memory can be an alternative step in future.

REFERENCES

- [1] D. Pereira *et al.*, "Inferencing on edge devices: A time and space aware co-scheduling approach," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, pp. 1–33, 2022.
- [2] Nvidia, "Cuda c++ programming guide," 2020.
- [3] Y. Xiang and H. Kim, "Pipelined data-parallel cpu/gpu scheduling for multi-dnn real-time inference," in *IEEE RTSS*, 2019.
- [4] H. Zhou *et al.*, "S³dnn: Supervised streaming and scheduling for gpu-accelerated real-time dnn workloads," in *IEEE RTAS*, 2018.
- [5] W. Kang *et al.*, "Lalarand: Flexible layer-by-layer cpu/gpu scheduling for real-time dnn tasks," in *IEEE RTSS*, 2021.
- [6] Z. Bo *et al.*, "Developing real-time scheduling policy by deep reinforcement learning," in *IEEE RTAS*, 2021.
- [7] H. Mao *et al.*, "Learning scheduling algorithms for data processing clusters," in *Proceedings of the ACM SIGCOMM*, 2019.
- [8] A. Ghose *et al.*, "Orchestration of perception systems for reliable performance in heterogeneous platforms," in *IEEE DATE*, 2021.
- [9] R. Huang *et al.*, "Yolo-lite: a real-time object detection algorithm optimized for non-gpu computers," in *IEEE Big Data*, 2018.
- [10] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *IEEE CVPR*, IEEE, 2017.
- [11] Saranya *et al.*, "Cyclist detection using tiny yolo v2," in *Springer, SocProS*, 2020.
- [12] Seidaliyeva *et al.*, "Detection of loaded and unloaded uav using deep neural network," in *IEEE IRC*, 2020.